Mansoura University
Faculty of Computers and Information
Department of Information Technology
Second Semester- 2025-2026

Software Reengineering

Grade: 4 SW

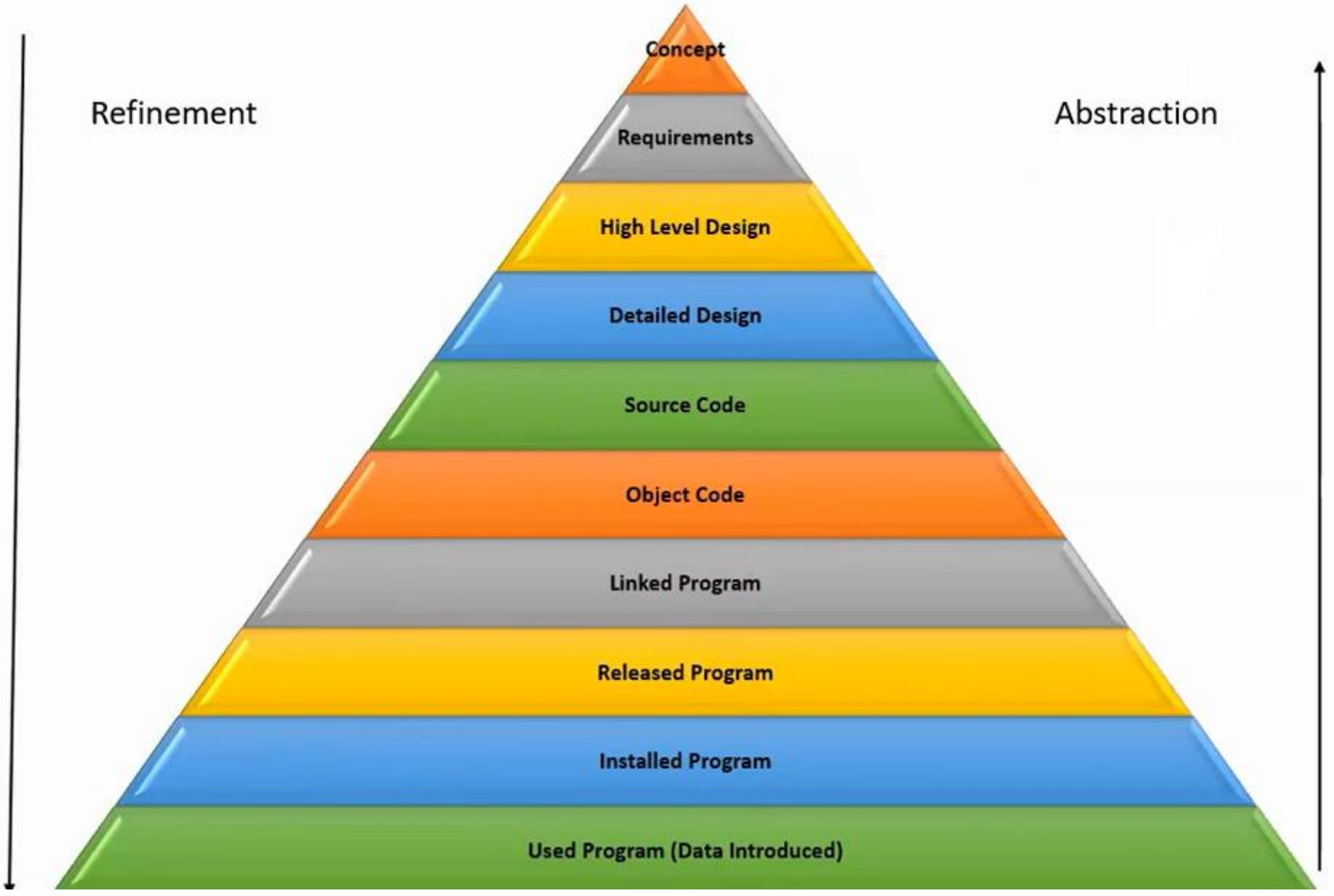
BY: Hala Ahmed

Chapter 3
Evolution and Maintenance Models

Outline

- General Idea
- Reuse Oriented Model
- The Staged Model for CSS
- The Staged Model for FLOSS
- Change Mini-Cycle Model
- IEEE/EIA 1219 Maintenance Process
- ISO/IEC 14764 Maintenance Process
- Software Configuration Management
  - Brief History
  - SCM Spectrum of Functionality
  - SCM Process
- Change Request Workflow

What is a Software



General Idea

- One traditional software development life cycle (SDLC) is shown in Figure, which comprises two discrete phases, namely:
  - development and
  - maintenance
- Maintenance approaching two-thirds of the product life-span.
- The percentages in Figure 3.1 indicate relative costs.

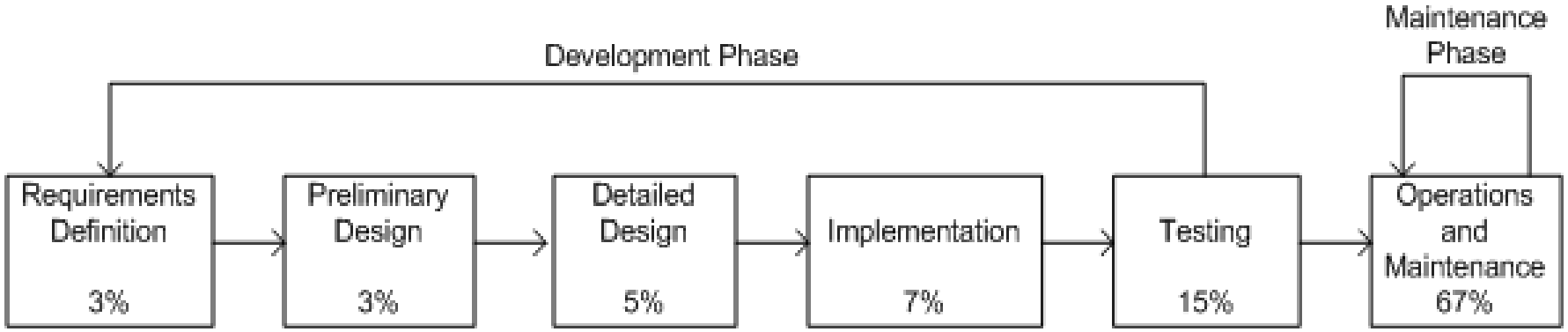


Figure 3.1: Traditional SDLC model

General Idea (Cont.)

Software maintenance has unique characteristics:

- **Constraints of an existing system:** Maintenance is performed on an operational system. Therefore, all modifications must be compatible with the constraints of the existing architecture, design, and code.
- **Shorter time frame:** A maintenance activity may span from a few hours to a few months, whereas software development may span one or more years.
- **Available test data:** In software development, test cases are designed from scratch, whereas software maintenance can select a subset of these test cases and execute them as regression tests.
- Software maintenance should have its own Software Maintenance Life Cycle (SMLC) model as it involves many unique activities.

Reuse Oriented Models

- One obtains a new version of an old system by modifying one or several components of the old system and possibly adding new components.
- Based on this concept, **three process models for maintenance** have been proposed by Basili:
  - **Quick fix model:** In this model, necessary changes are quickly made to the code and then to the accompanying documentation.
  - **Iterative enhancement model:** In this model, first changes are made to the highest level documents. Eventually, changes are propagated down to the code level.
  - **Full reuse model:** In this model, a new system is built from components of the old system and others available in the repository.

Reuse Oriented Models

In **Quick Fix Model**, as illustrated in Figure 3.2

i. source code is modified to fix the problem;

ii. necessary changes are made to the relevant documents; and

iii. the new Acode is recompiled to produce a new version.

Often changes to the source code are made with no prior investigation such as analysis of impact of the changes, ripple effects of the changes, and regression testing.

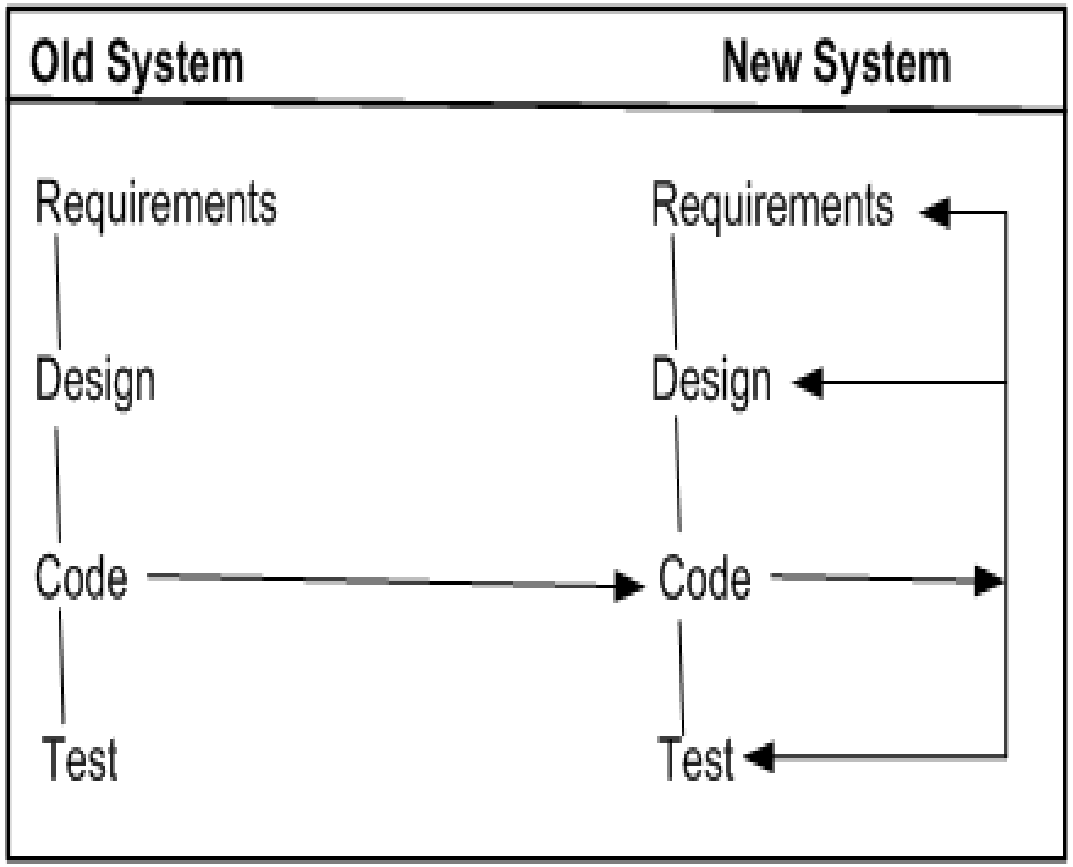


Figure 3.2 The quick fix model.

Reuse Oriented Models (Cont.)

Iterative Vs. Incremental

- The terms **iteration** and **increment** are liberally used when discussing iterative and incremental development.
- However, they are not synonyms in the field of software engineering.
- **Iteration** implies that a process is basically cyclic, thereby meaning that the activities of the process are repeatedly executed in a structured manner.
- **Iterative** development is based on scheduling strategies in which time is set aside to improve and revise parts of the system under development.
- **Incremental** development is based on staging and scheduling strategies in which parts of the system are developed at different times and/or paces, and integrated as they are completed.

Reuse Oriented Models (Cont.)

The **iterative enhancement model**, explained in Figure 3.3, shows how changes flow from the very top level documents to the lowest-level documents.

The model works as follows:

- It begins with the existing system's artifacts, namely, requirements, design, code, test, and analysis documents.
- It revises the highest-level documents affected by the changes and propagates the changes down through the lower-level documents.
- The model allows maintainers to redesign the system, based on the analysis of the existing system.

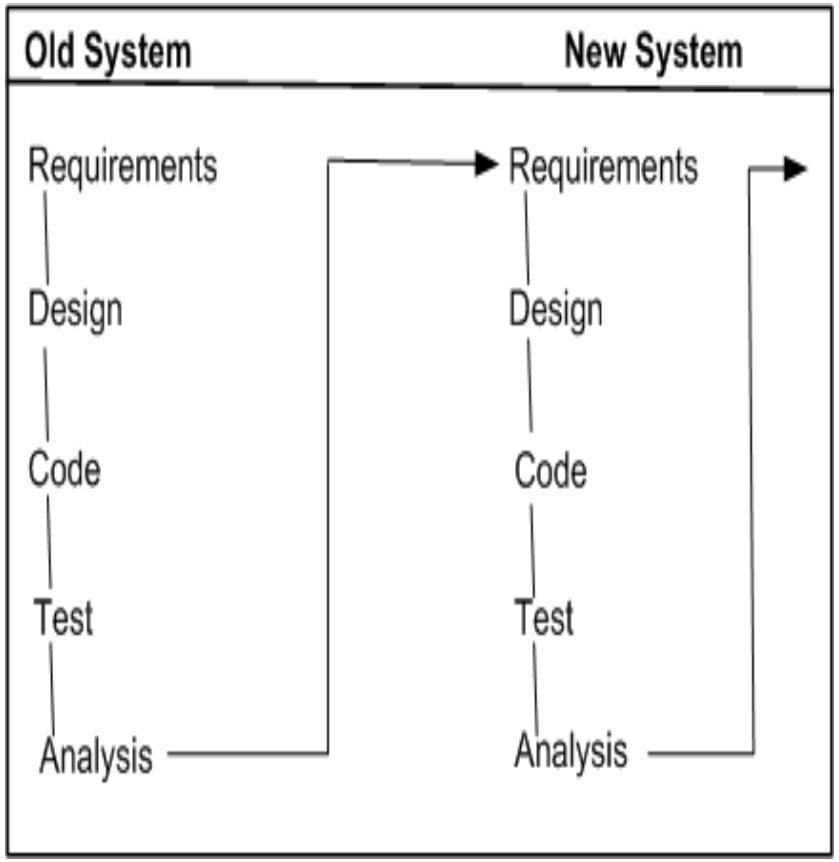


Figure 3.3 The iterative enhancement model

Reuse Oriented Models (Cont.)

- The main assumption in this model is the availability of a repository of artifacts describing the earlier versions of the systems.
- In the **full reuse model**, reuse is explicit and the following activities are performed:
  - identify the components of the old system that are candidates for reuse
  - understand the identified system components.
  - modify the old system components to support the new requirements.
  - integrate the modified components to form the newly developed system.

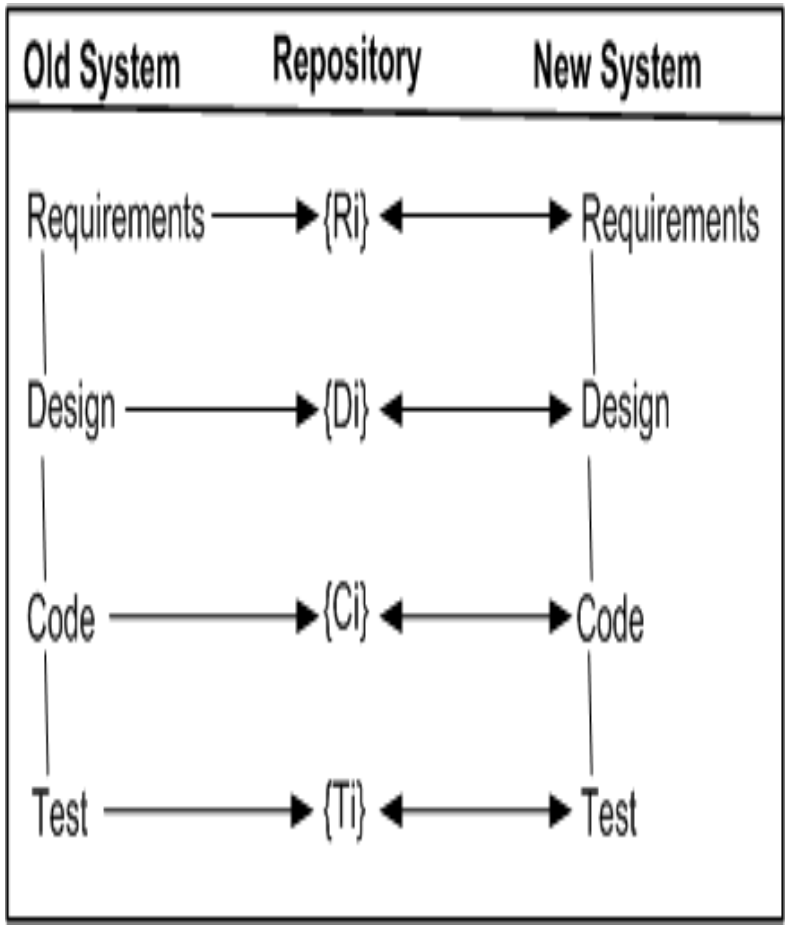


Figure 3.4 The full reuse model

The Staged Model for CSS

- Rajlich and Bennett have presented a descriptive model of software evolution called the staged model of maintenance and evolution.
- Its primary objective is at improving understanding of how long-lived software evolves, rather than aiding in its management.
- Their model divides the lifespan of a typical system into **four stages:**
  - **Initial development** – The first delivered version is produced. Knowledge about the system is fresh and constantly changing. In fact, change is the rule rather than the exception. Eventually, an architecture emerge and stabilizes.
  - **Evolution** – Simple changes are easily performed, and more major changes are possible too, albeit the cost and risk are now greater than in the previous stage. Knowledge about the system is still good, although many of the original developers will have moved on. For many systems, most of its lifespan is spent in this phase.

The Staged Model for CSS

- **Servicing** – The system is no longer a key asset for the developers, who concentrate mainly on maintenance tasks rather than architectural or functional change. Knowledge about the system has lessened and the effects of change have become harder to predict. The costs and risks of change have increased significantly.
- **Phase out** – It has been decided to replace or eliminate the system entirely, either because the costs of maintaining the old system have become prohibitive or because there is a newer solution waiting in the pipeline. An exit strategy is devised and implemented, often involving techniques such as wrapping and data migration. Ultimately, the system is shut down.

The Staged Model for CSS

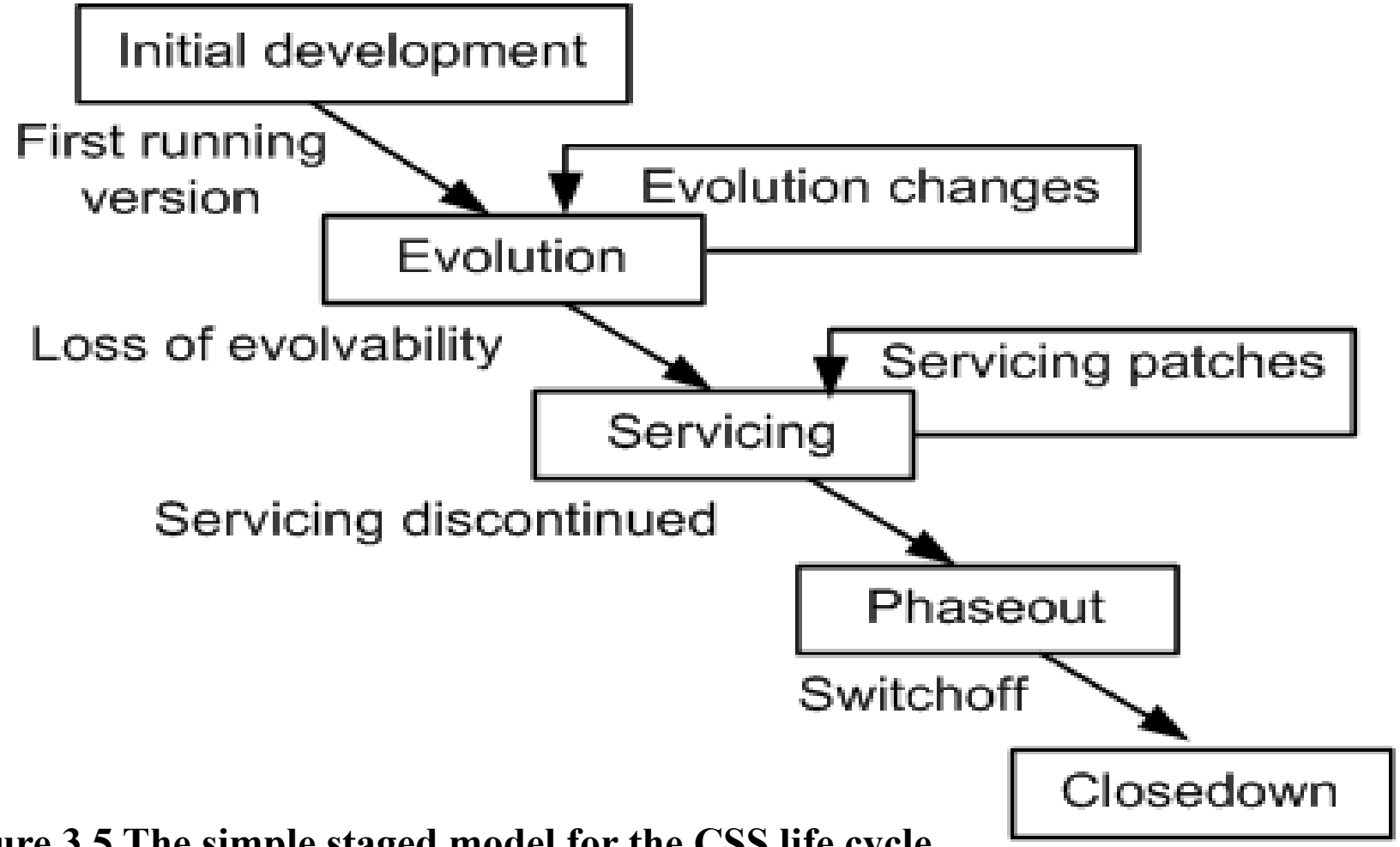


Figure 3.5 The simple staged model for the CSS life cycle

The Staged Model for CSS

The evolution process is the backbone of the model.

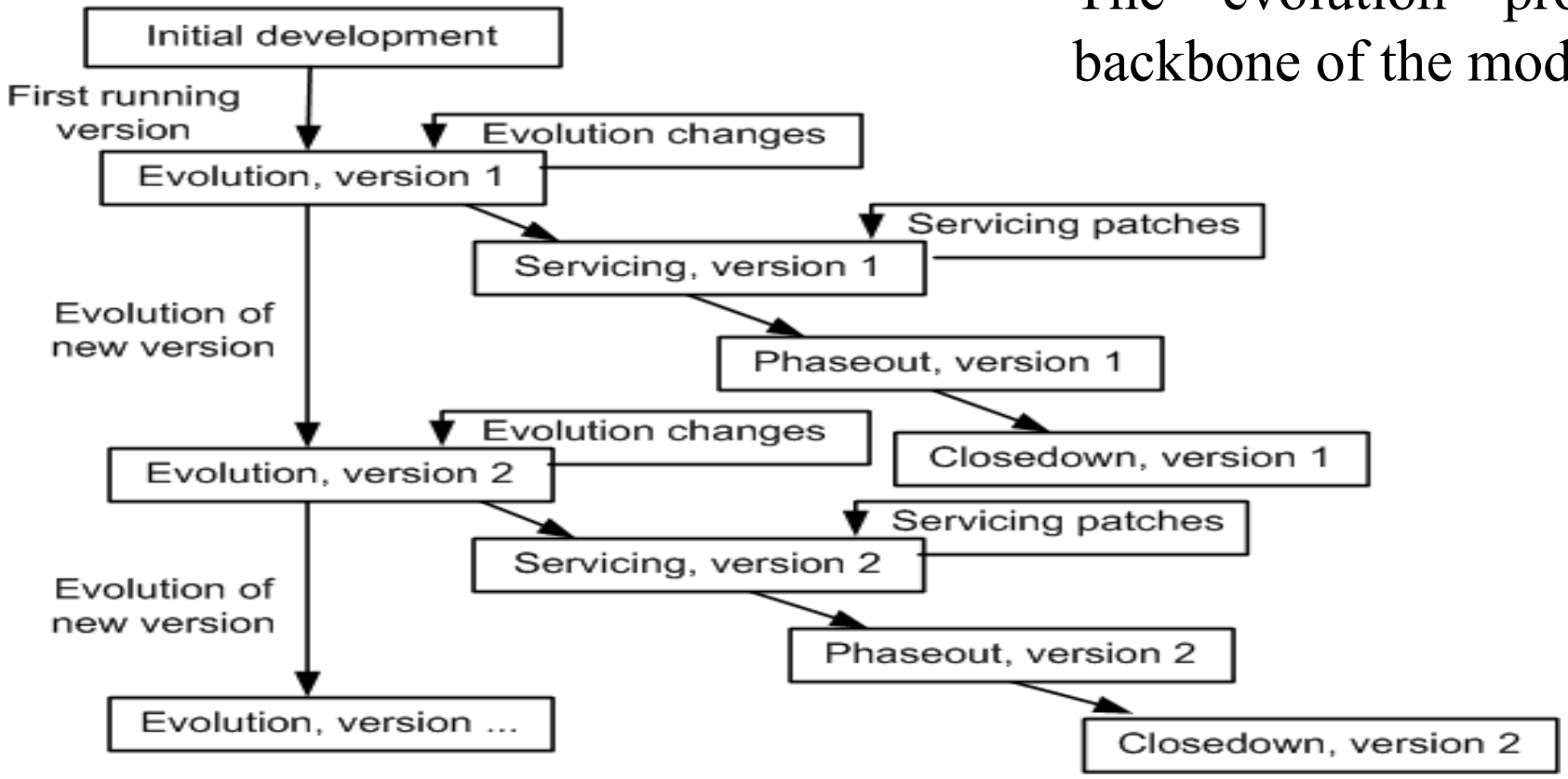


Figure 3.6 The versioned staged model for the CSS life cycle

The Staged Model for FLOSS

- Three major differences are identified between CSS systems and FLOSS systems.
- In Figure 3.7, the rectangle with the label “Initial development” has been visually highlighted because it can be the only initial development stage in the evolution of FLOSS systems. In other words, it does not have any evolution track for FLOSS system.
- With some systems that were analyzed, after a transition from Evolution to Servicing, a new period of evolution was observed. This possibility is depicted in Figure as a broken arc from the Servicing stage to the Evolution stage.
- In general, the active developers of FLOSS systems get frequently replaced by new developers. Therefore, the dashed line in Figure exhibits this possibility of a transition from Phaseout stage to Evolution stage.

The Staged Model for FLOSS(Cont.)

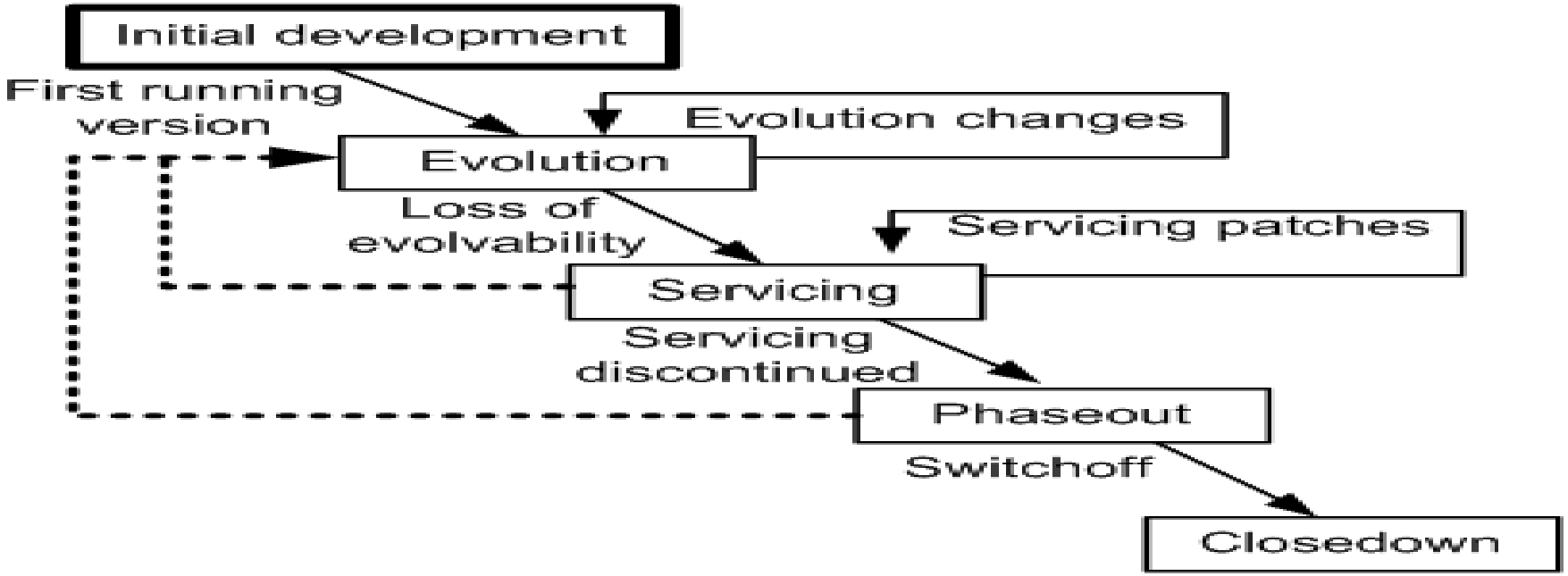


Figure 3.7 The staged model for FLOSS system

Change Mini-Cycle Model

- **Software change** is **a process** that may introduce **new requirements to the existing system or** may need to **alter the software system** if requirements are not correctly implemented.
- **In order to capture this**, an evolutionary model known as **change mini-cycle** was proposed by Yau et al. in the late seventies.

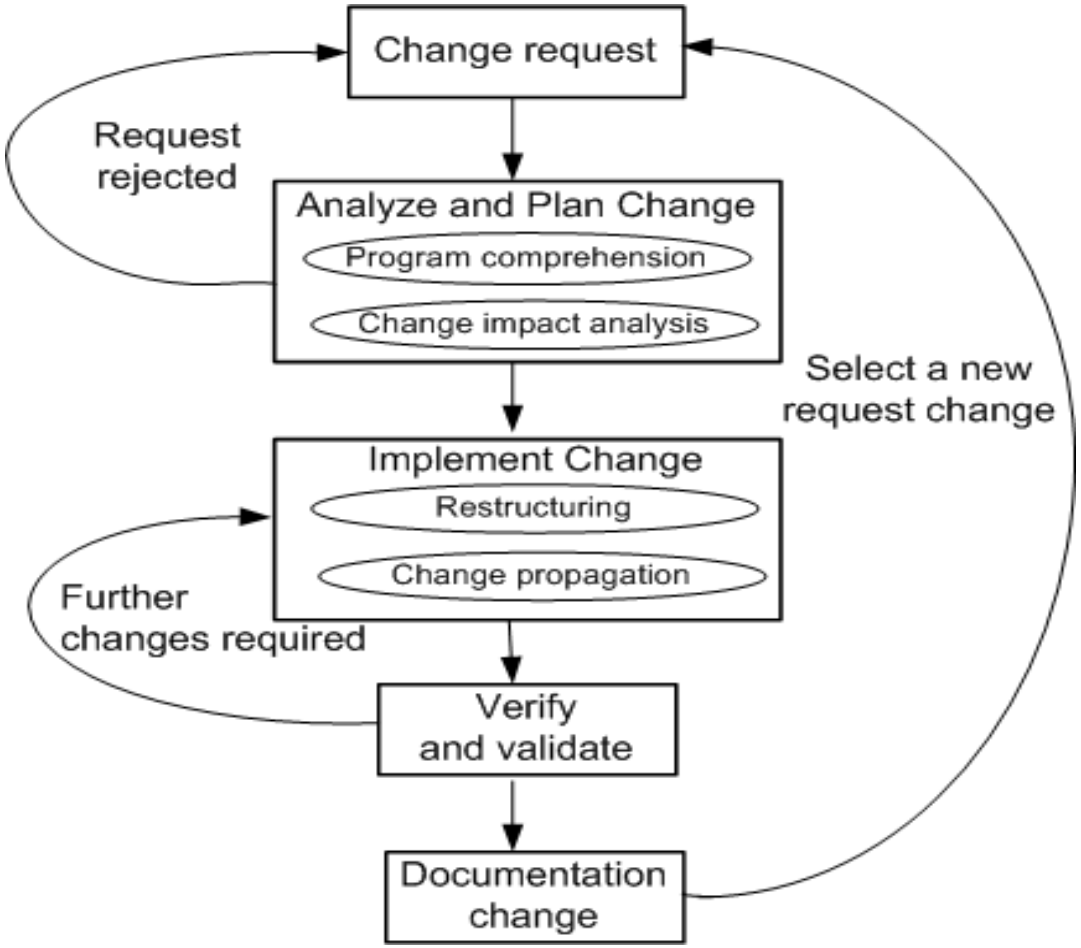


Figure 3.8 The change mini-cycle

Software Maintenance Standard

- IEEE and ISO have both addressed s/w maintenance processes.
- IEEE/EIA 1219 and ISO/IEC 14764 as a part of ISO/IEC12207 (life cycle process).
- IEEE/EIA 1219 organizes the maintenance process in seven phases:
  - problem identification, analysis, design, implementation, system test, acceptance test and delivery.
- ISO/IEC 14764 describes s/w maintenance as an iterative process for managing and executing software maintenance activities.

Software Maintenance Standard(Cont.)

- The activities which comprise the maintenance process are:
  - process implementation, problem and modification analysis, modification implementation, maintenance review/acceptance, migration and retirement.
- Each of these maintenance activity is made up of tasks. Each task describes a specific action with inputs and outputs.

IEEE/EIA 1219 Maintenance Process

The standard focuses on a seven-phases:

- Problem Identification.
- Analysis.
- Design.
- Implementation.
- System Test.
- Acceptance Test.
- Delivery.

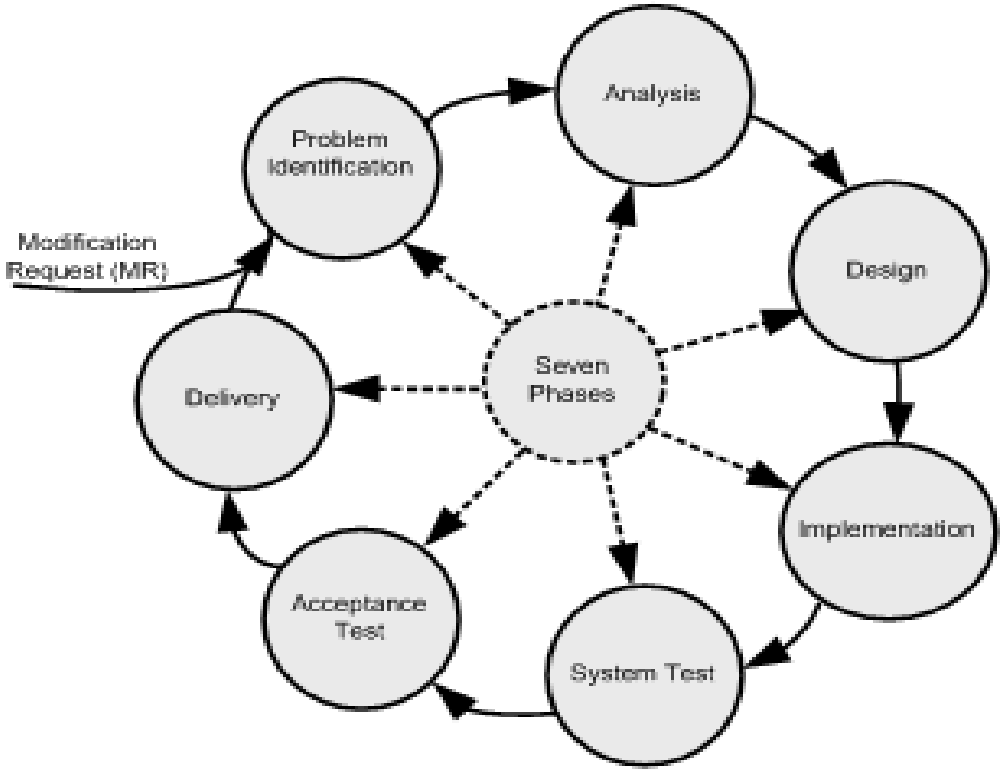


Figure 3.9 Seven phases of IEEE maintenance

IEEE/EIA 1219 Maintenance Process(Cont.)

Each of the seven activities has five associated attributes as follows:

- **Activity definition:** This refers to the implementation process of the activity.
- **Input:** This refers to the items that are required as input to the activity.
- **Output:** This refers to the items that are produced by the activity.
- **Control:** This refers to those items that provide control over the activity.
- **Metrics:** This refers to the items that are measured during the execution of the activity.

IEEE/EIA 1219 Maintenance Process(Cont.)

- A request for change to the software is normally made by the users of the software system or the customers, and it starts the maintenance process.
- The request for change (CR) is submitted in the form of a modification request (MR) for a correction or for an enhancement. MR & CR are used interchangeably.
- **Activities included in this phase are as follows:**
  i. reject or accept the MR,
  ii. identify and estimate the resources needed to change the system; and
  iii. put the MR in a batch of changes scheduled for implementation.

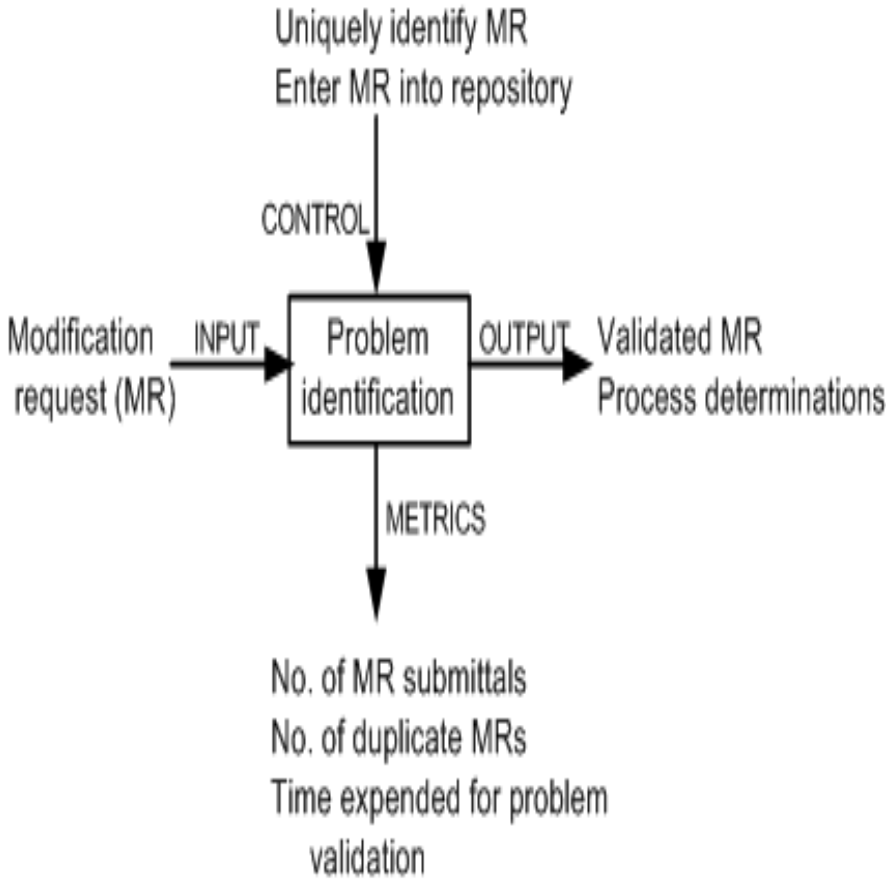


Figure 3.10 Problem identification phase

IEEE/EIA 1219 Maintenance Process

The process is viewed to have two major components:

(i) feasibility analysis.

(ii) detailed analysis.

- First, feasibility analysis is performed to:
  - determine the impact of the change;
  - investigate other possible solutions including prototyping;
  - assess both short-term and long-term costs; and
  - determine the benefits of making the change.
- The second phase identifies: (i) firm modification requirements; (ii) the software components involved; (iii) an overall test strategy; and (iv) an implementation plan.

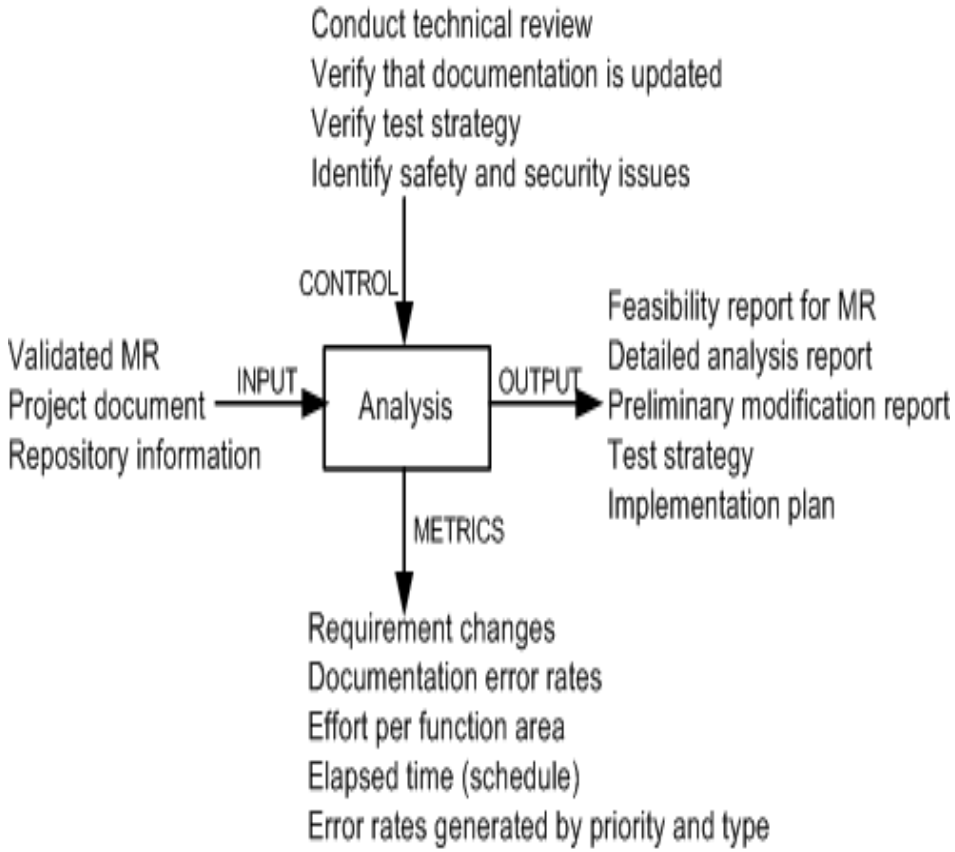


Figure 3.11 Analysis phase

IEEE/EIA 1219 Maintenance Process

Activities of this phase are as follows:

i. identify the affected software components.

ii. modify the software components.

iii. document the changes.

iv. create a test suite for the new design.

v. select test cases for regression testing.

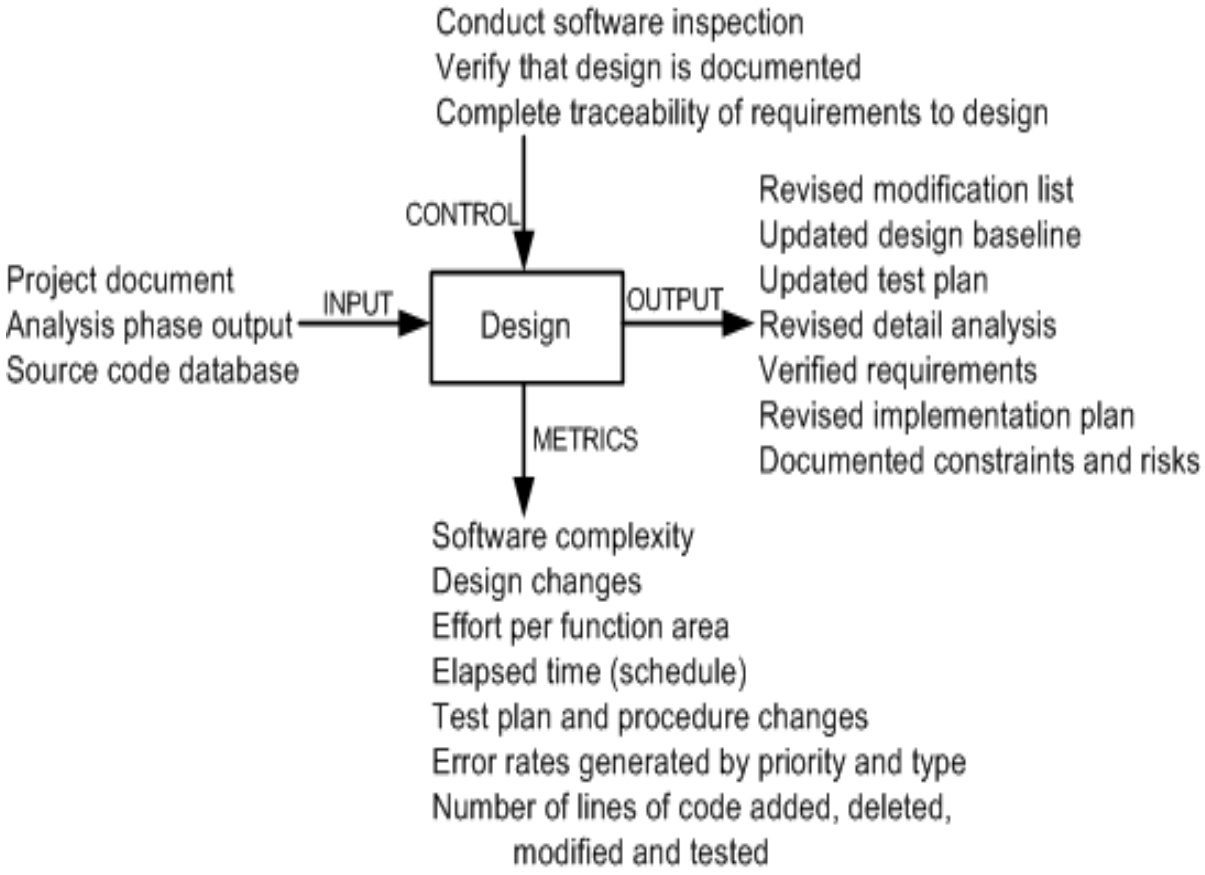


Figure 3.12 Design phase

IEEE/EIA 1219 Maintenance Process

The activities executed in this phase are:

- writing new code and performing unit testing,
- integrating changed code,
- conducting integration and regression testing,
- performing risk analysis, and
- reviewing the system for test-readiness.

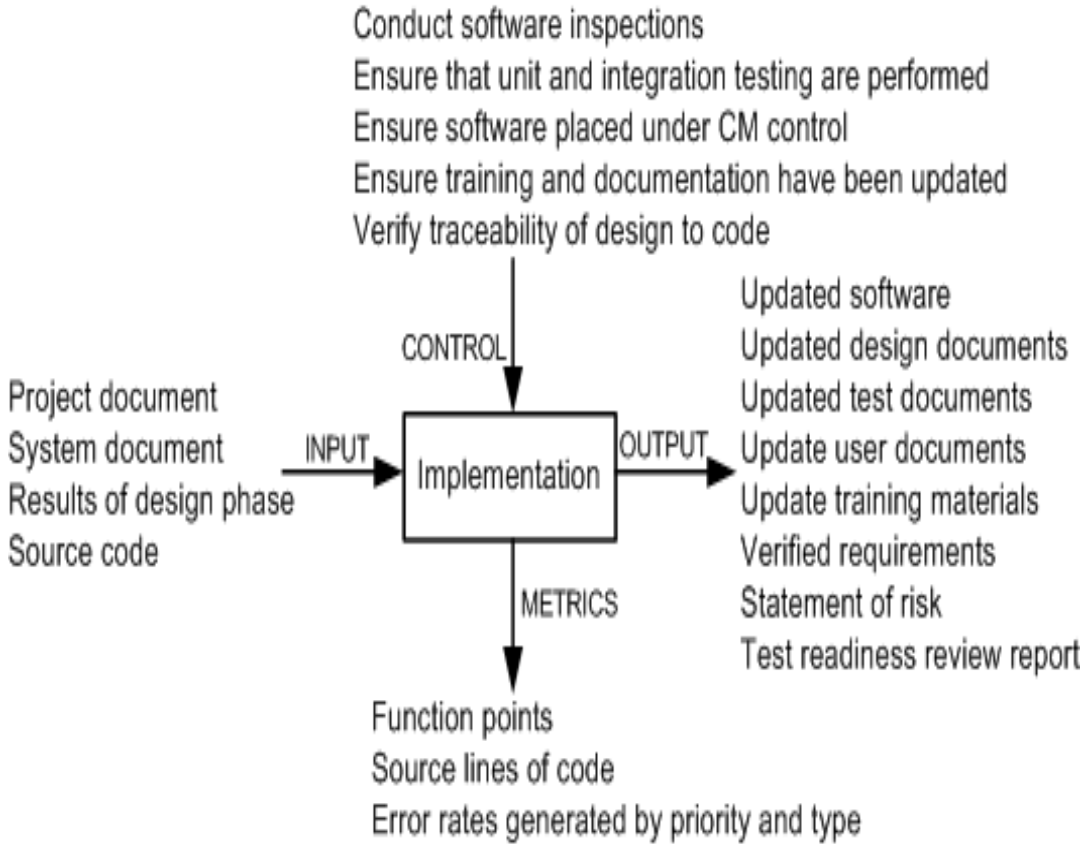


Figure 3.13 Implementation phase

IEEE/EIA 1219 Maintenance Process

- In this phase tests are performed on the full system to ensure that the modified system complies with the original requirements as well as the new modifications.

- System-level testing comprises a broad spectrum of testing activities: functionality testing, robustness testing, stability testing, load testing, performance testing, security testing, and regression testing.

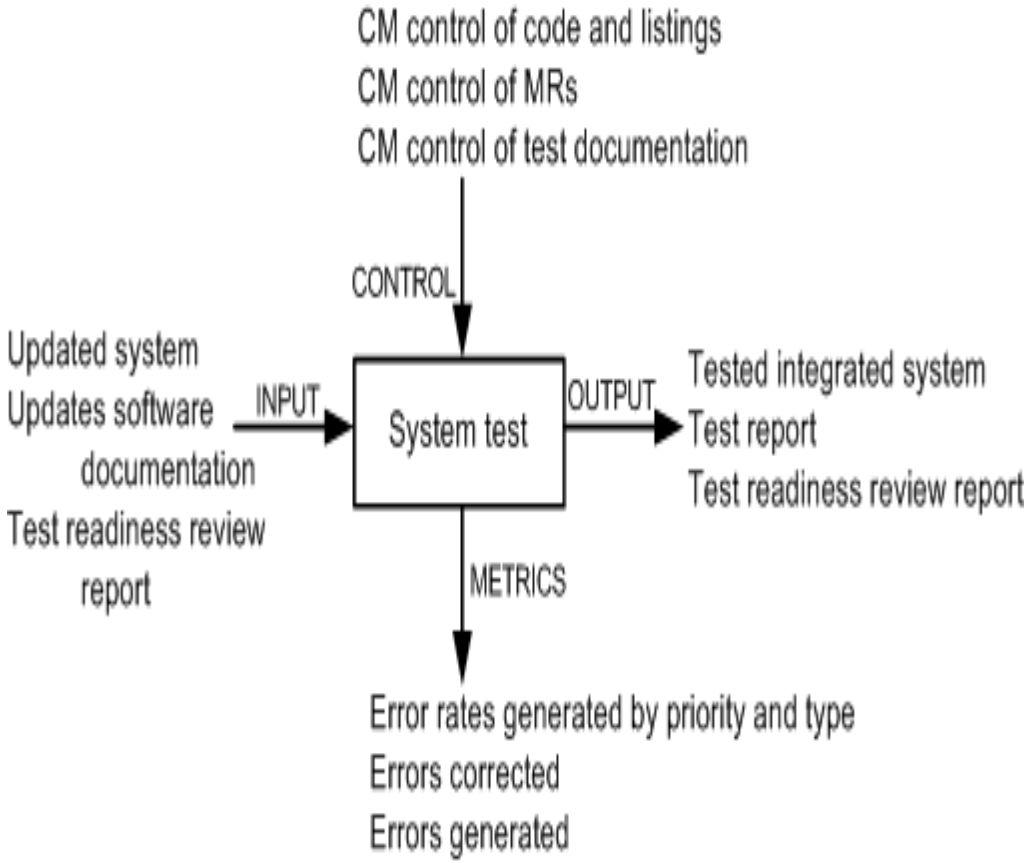


Figure 3.14 System test phase

IEEE/EIA 1219 Maintenance Process

- Acceptance testing is performed on a completely integrated system, and it involves customers, users, or their representatives.
- The main objective of acceptance testing is to assess the overall quality of the system, rather than actively identify defects.
- An important concept in acceptance testing is the customer's expectation from the system.

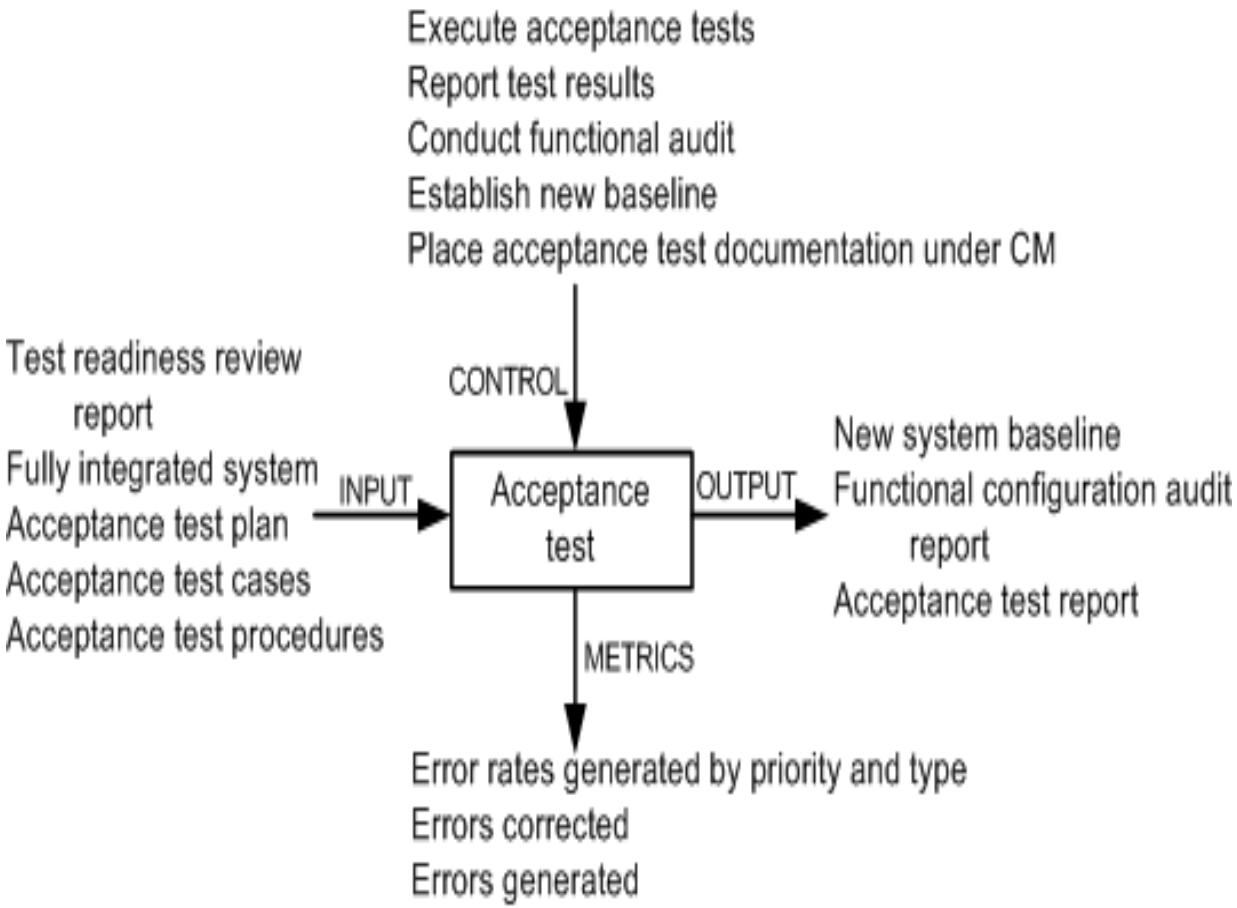


Figure 3.15 Acceptance test phase

IEEE/EIA 1219 Maintenance Process

- In this phase, the changed system is released to customers for installation and operation.
- Included in this phase are the following activities: notify the user community, perform installation and training, and develop an archival version of the system for backup.

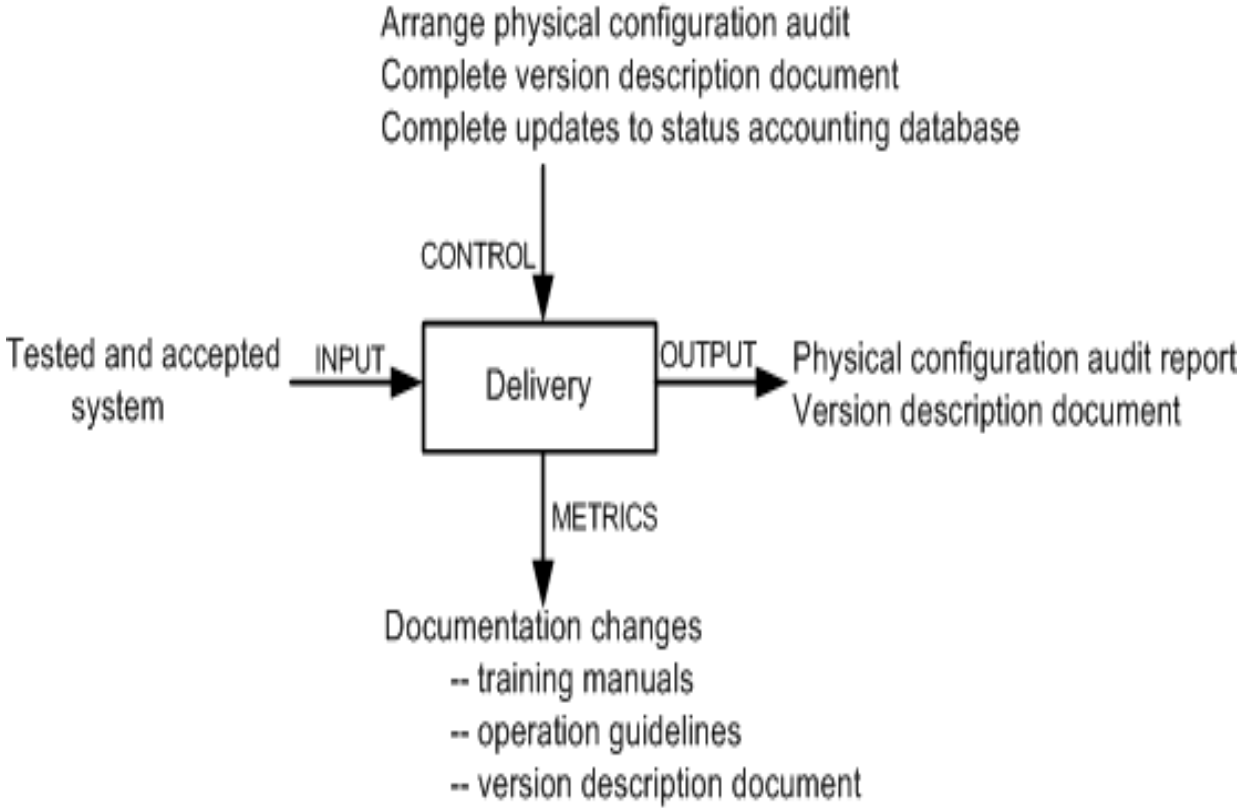


Figure 3.16 Delivery phase

ISO/IEC 14764 Maintenance Process

- ISO/IEC 14764 is an international standard for software maintenance.
- It describes maintenance using the same concepts as IEEE/EIA 1219 except that they are depicted slightly differently.
- The basic structure of an ISO process is made up of activities, and an activity is made up of tasks.

ISO/IEC 14764 Maintenance Process

- Upon an activation of the maintenance process, plans and procedures are developed and resources are allocated to carry out maintenance.
- In response to a change request, code is modified in conjunction with the relevant documentation.
- Modification of the running software without losing the system's integrity is considered to be the overall objective of maintenance.

ISO/IEC 14764 Maintenance Process

The maintenance process comprises the following high level activities:

1. Process Implementation.
2. Problem and Modification Analysis.
3. Modification Implementation.
4. Maintenance Review and Acceptance.
5. Migration.
6. Retirement.

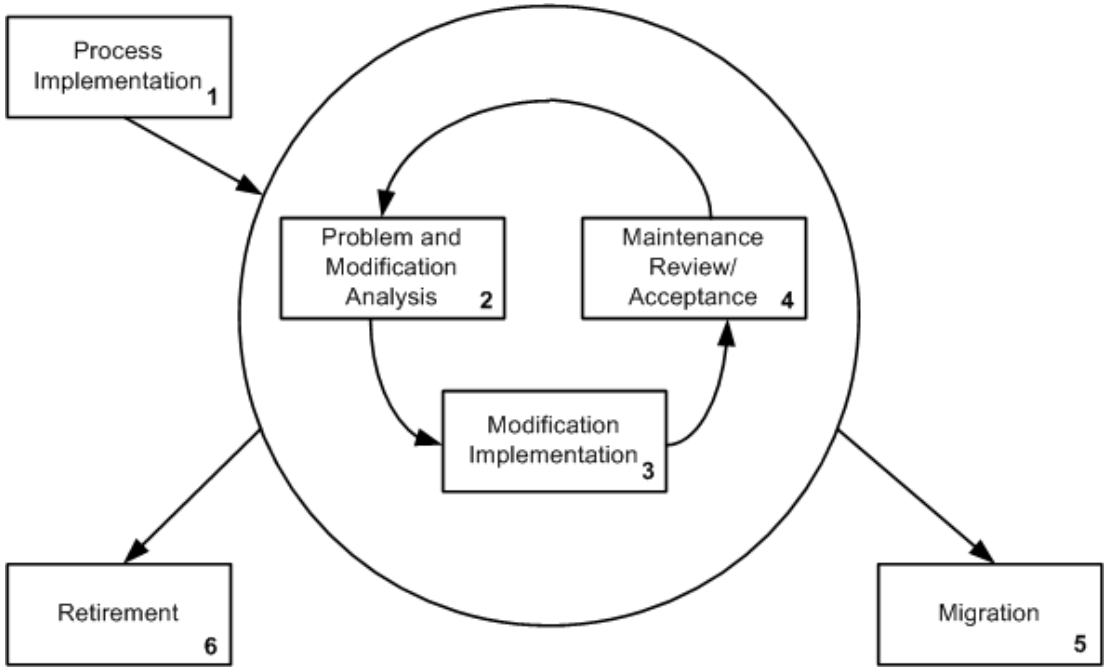


Figure 3.17 ISO/IEC 14764 iterative maintenance process

ISO/IEC 14764 Maintenance Process

The process implementation activity consists of three major tasks:

- Maintenance plan.
- Modification requests (MR/CR).
- Configuration management.

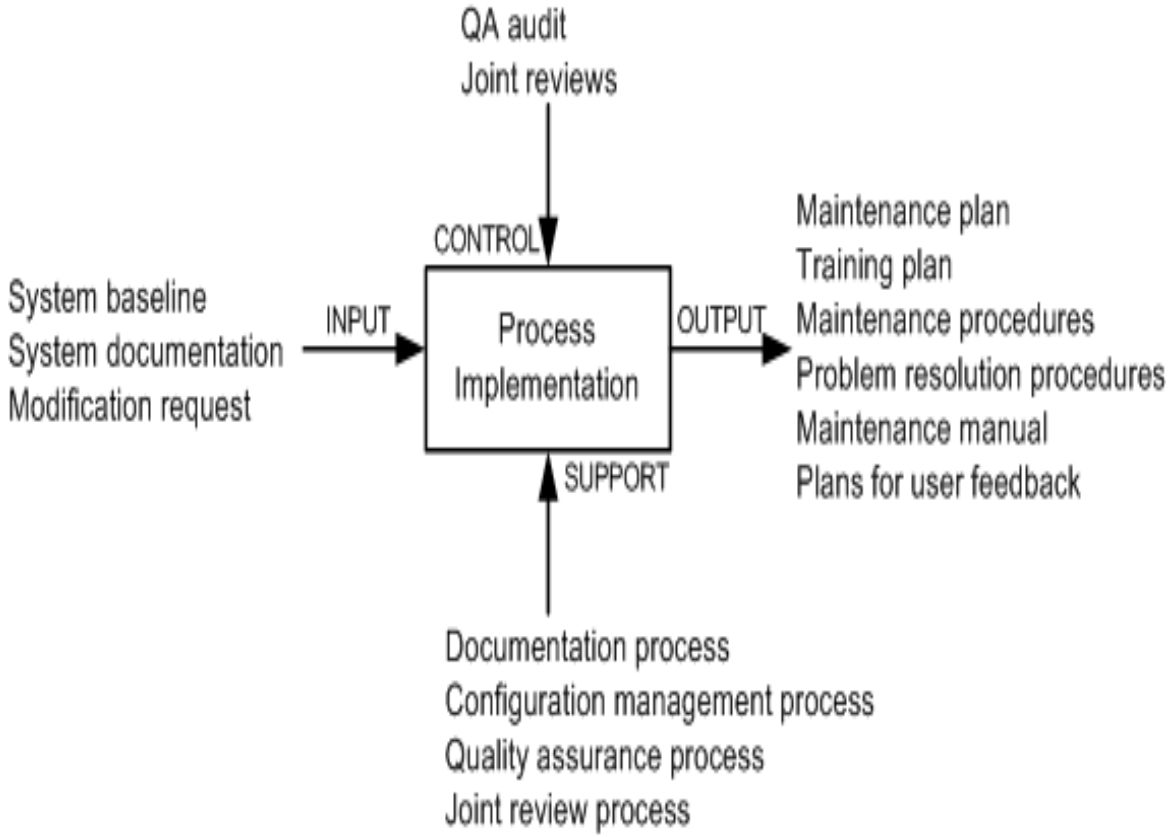


Figure 3.18 Process implementation activity

ISO/IEC 14764 Maintenance Process

TABLE 3.2 Modification Request Task Steps

1. Decide whether or not the maintainer is adequately staffed to make the proposed changes.
2. Decide whether or not the maintenance program has received adequate budget.
3. Decide whether or not enough resources are available and whether the proposed change will effect some current or future projects.
4. Determine the operational issues to be considered.
5. Determine handling priority.
6. Classify the type of maintenance.
7. Determine the impact to current and future users.
8. Determine safety and security implications.
9. Identify ripple effects.
10. Determine any hardware or software constraints that may result from the proposed changes.
11. Estimate the values of the benefits of making the changes.
12. Determine the impact on existing schedules.
13. Document the risks resulting from the impact analysis.
14. Estimate the evaluation to be performed.
15. Estimate the cost of management to execute the modification.
16. Place developed artifacts under CM.

ED PROOFS

ISO/IEC 14764 Maintenance Process

- **The problem and modification analysis activity comprises five tasks:**
  - Modification request analysis.
  - Verification.
  - Options.
  - Documentation.
  - Approval.

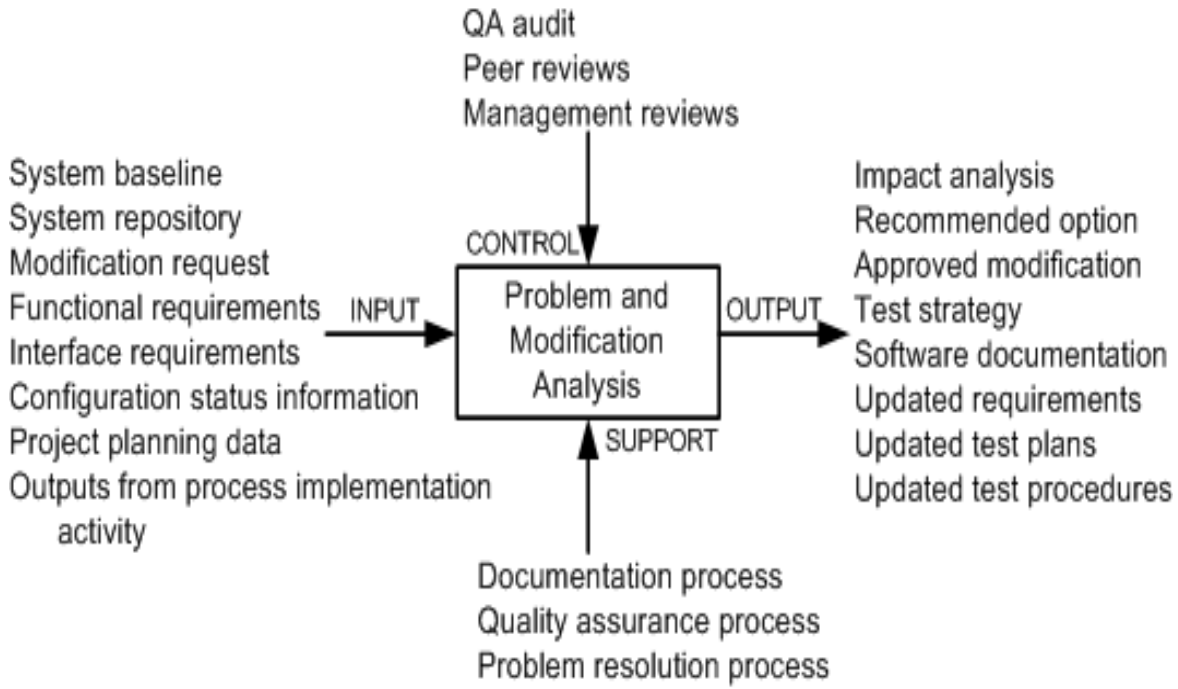


Figure 3.19 Problem modification activity

TABLE 3.3 Option Task Steps

1. The MR is assigned a work priority.
2. Explore a work-around for the problem. If a work-around exists, provide it to the user.
3. Identify concrete requirements for the planned modification.
4. Calculate the magnitude and size of the planned modification.
5. Identify a variety of options to execute the planned modification.
6. Estimate the impacts of the options on the users and system hardware.
7. Analyze the risks of each option.
8. Document the outcomes of risk analysis for each of the proposed options.
9. Develop a widely acceptable plan to implement the modification.

TABLE 3.4 Documentation Task Steps

1. Ensure result analyses have been completed and documentations updated. If documentations do not exist, develop new documentation.
2. For accuracy, review the planned strategy to perform tests and review the schedule.
3. Review resource estimates for accuracy.
4. Revise the database for storing accounting status.
5. Describe a procedure to decide whether or not to approve the MR.

ISO/IEC 14764 Maintenance Process

In this activity, maintainers:

1. identify the items to be modified, and
2. execute a development process to actually implement the modifications.

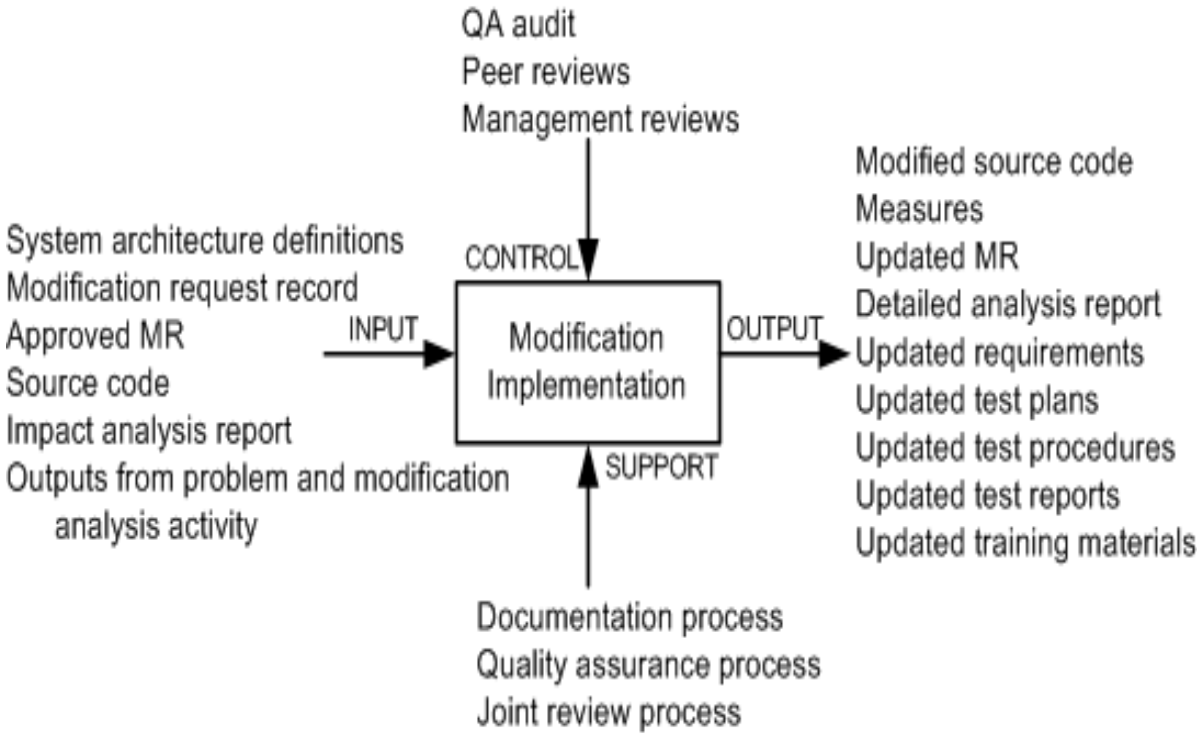


Figure 3.20 Modification implementation activity

ISO/IEC 14764 Maintenance Process

By means of this activity, it is ensured that:

1. the changes made to the software are correct, and
2. changes are made to the software according to accepted standards and methodologies.

The activity is augmented with the following processes:

1. a process for quality management,
2. a process to verify the product,
3. a process to validate the product, and
4. a process to review the product.

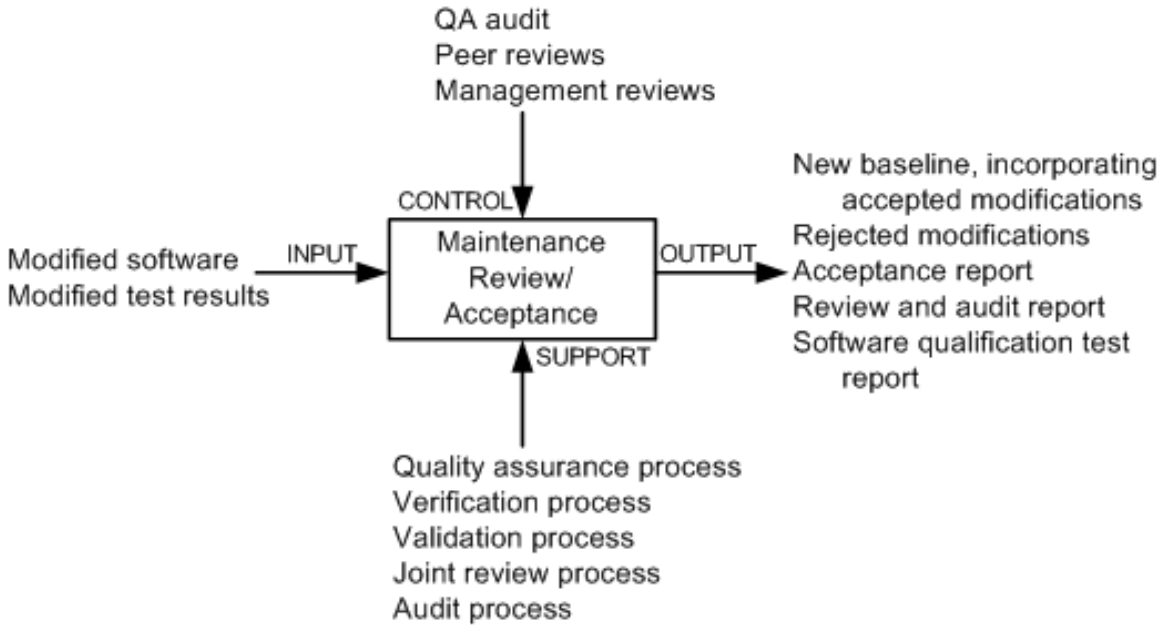


Figure 3.21 Maintenance review/acceptance activity

ISO/IEC 14764 Maintenance Process

TABLE 3.5 Review and Approval Task Steps

Review Task Steps

1. Track the MRs from requirement specification to coding.
2. Ensure that the code is testable.
3. Ensure that the code conforms to coding standards.
4. Ensure that only the required software components were changed.
5. Ensure that the new code is correctly integrated with the system.
6. Ensure that documentations are accurately updated.
7. CM personnel build software items for testing.
8. Perform testing by an independent test organization.
9. Perform system test on a fully integrated system.
10. Develop test report.

Approval Task Steps

1. Obtain quality assurance approval.
2. Verify that the process has been followed.
3. CM prepares the delivery package.
4. Conduct functional and physical configuration audit.
6. Notify operators.
7. Perform installation and training at the operator's facility.

ISO/IEC 14764 Maintenance Process

Migration is effected in two broad phases:

1. identify the actions required to achieve migration, and
2. design and document the concrete steps to be executed to effect migration.

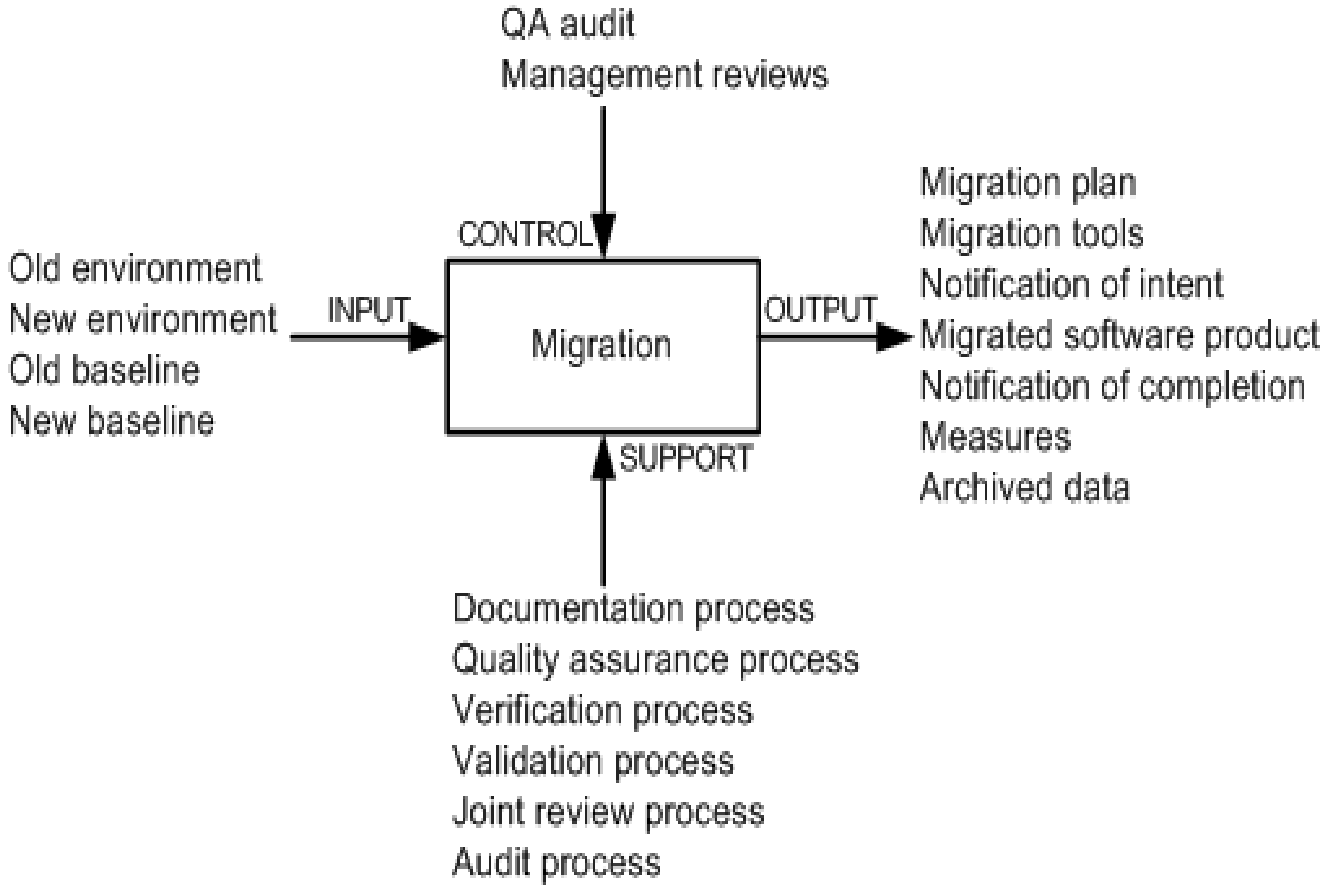


Figure 3.22 Migration activity

ISO/IEC 14764 Maintenance Process

TABLE 3.6 Migration Plan Task Steps

1. Analyze the requirements for migration.
2. Perform an impact analysis of migrating the software system.
3. Make a schedule to execute migration.
4. Determine all requirements for data collection to perform post-operation review.
5. Identify and record the migration effort.
6. Identify and reduce risks.
7. Identify the required tools to support migration.
8. Determine how the old environment is going to be supported.
9. Acquire and/or design new tools to support migration.
10. Partition software products and data for conversion in an incremental manner.
11. Prioritize the activities involving data conversion and software products.
12. Execute software products and data conversions.
13. Perform migration of software products and data to the new environment.
14. Operate the migrated system and the old system in parallel as much as possible.
15. Perform testing to ensure the success of migration.
16. Should there be a need, continue to provide support for the old environment.

ISO/IEC 14764 Maintenance Process

TABLE 3.7 Operation and Training Task Steps

Parallel Operations Task Steps

1. Survey the site.
2. Install hardware equipment.
3. Install the software system.
4. Run basic tests to ensure that hardware and software have been correctly installed.
5. Run both the new and old systems in parallel, under the desired operational load.
6. Gather data from the old and the new systems.
7. Analyze the collected data.

Training Task Steps

1. Identify the requirements for migration training.
2. Schedule the requirements for migration training.
3. Review the migration training.
4. Update the plan to provide training.

ISO/IEC 14764 Maintenance Process

This activity comprises five tasks:

- Retirement plan
- Notification of intent
- Implement parallel operations and training
- Notification of completion
- Data archival

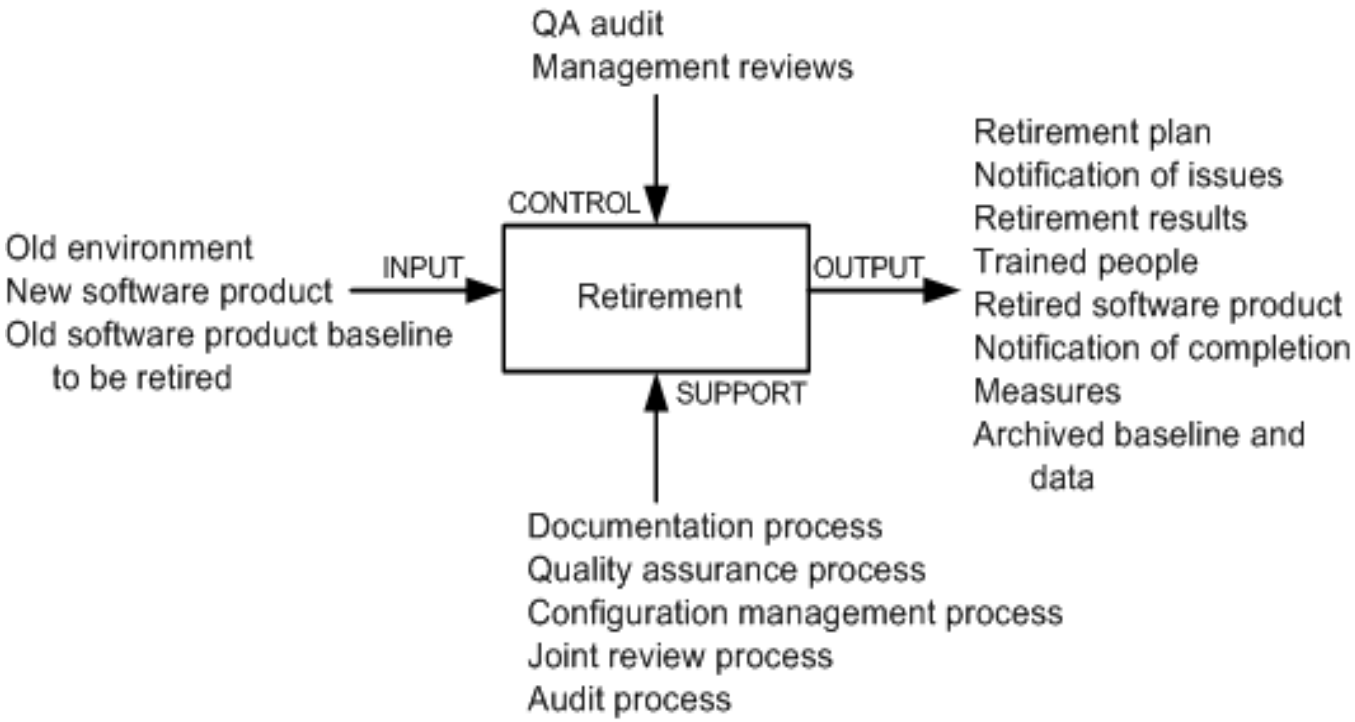


Figure 3.23 Retirement activity

ISO/IEC 14764 Maintenance Process

TABLE 3.8 Retirement Plan Task Steps

1. Analyze the requirements to retire the systems.
2. Determine what impacts the retiring software will have.
3. Identify a product that will replace the software to be retired.
4. Make a schedule to retire the software.
5. Determine the need for residual support in the future.
6. Identify and describe the retirement effort.

Thank you